

SOLUTIONS

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **SingletonPatternExample**.
2. **Define a Singleton Class:**
 - Create a class named **Logger** that has a private static instance of itself.
 - Ensure the constructor of **Logger** is private.
 - Provide a public static method to get the instance of the **Logger** class.
3. **Implement the Singleton Pattern:**
 - Write code to ensure that the **Logger** class follows the Singleton design pattern.
4. **Test the Singleton Implementation:**
 - Create a test class to verify that only one instance of **Logger** is created and used across the application.

SOLN:

```
class Logger {
    private static Logger obj=new Logger();
    private Logger(){}
    public static Logger getInstance(){
        return obj;
    }
}

class Main {
    public static void main(String[] args) {

        Logger l1 = Logger.getInstance();
        Logger l2 = Logger.getInstance();

        // Verify both references point to the same object
        System.out.println(l1 == l2);
    }
}
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. **Create a New Java Project:**

- Create a new Java project named **FactoryMethodPatternExample**.

2. **Define Document Classes:**

- Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.

3. **Create Concrete Document Classes:**

- Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.

4. **Implement the Factory Method:**

- Create an abstract class **DocumentFactory** with a method **createDocument()**.
- Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.

5. **Test the Factory Method Implementation:**

- Create a test class to demonstrate the creation of different document types using the factory method.

SOLN:

```
1 interface Document {
2     void open();
3 }
4 class WordDocument implements Document{
5     public void open(){
6         System.out.println("Opening Word Document");
7     }
8 }
9 class PDFDocument implements Document{
10     public void open(){
11         System.out.println("Opening PDF Document");
12     }
13 }
14 class ExcelDocument implements Document{
15     public void open(){
16         System.out.println("Opening Excel Document");
17     }
18 }
19 abstract class DocumentFactory {
20     abstract Document createDocument();
21 }
22 class WordDocumentFactory extends DocumentFactory {
23     Document createDocument(){
24         return new WordDocument();
25     }
26 }
27 class PDFDocumentFactory extends DocumentFactory {
28     Document createDocument(){
29         return new PDFDocument();
30     }
31 }
32 class ExcelDocumentFactory extends DocumentFactory {
33     Document createDocument(){
34         return new ExcelDocument();
35     }
36 }
37 class Main {
38     public static void main(String[] args) {
39
40         DocumentFactory factory =
41             new WordDocumentFactory();
42     }
```

Exercise 3: Implementing the Builder Pattern

Scenario:

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **BuilderPatternExample**.
2. **Define a Product Class:**
 - Create a class **Computer** with attributes like **CPU**, **RAM**, **Storage**, etc.
3. **Implement the Builder Class:**
 - Create a static nested Builder class inside Computer with methods to set each attribute.
 - Provide a **build()** method in the Builder class that returns an instance of Computer.
4. **Implement the Builder Pattern:**
 - Ensure that the **Computer** class has a private constructor that takes the **Builder** as a parameter.
5. **Test the Builder Implementation:**
 - Create a test class to demonstrate the creation of different configurations of Computer using the Builder pattern.

```
class Computer {  
  
    private String cpu;  
    private String ram;  
    private String storage;  
  
    private Computer(Builder b) {  
        cpu = b.cpu;  
        ram = b.ram;  
        storage = b.storage;  
    }  
  
    static class Builder {  
  
        private String cpu;  
        private String ram;  
        private String storage;  
  
        public Builder setCpu(String cpu) {  
            this.cpu = cpu;  
            return this;  
        }  
  
        public Builder setRam(String ram) {  
            this.ram = ram;  
            return this;  
        }  
    }  
}
```

```
        public Builder setStorage(String storage) {  
            this.storage = storage;  
            return this;  
        }  
  
        public Computer build() {  
            return new Computer(this);  
        }  
  
        public void display() {  
            System.out.println("CPU: " + cpu);  
            System.out.println("RAM: " + ram);  
            System.out.println("Storage: " + storage);  
        }  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
  
        Computer c1 = new Computer.Builder()  
            .setCpu("Intel i5")  
            .setRam("8GB")  
            .setStorage("512GB SSD")  
            .build();  
  
        Computer c2 = new Computer.Builder()  
            .setCpu("Intel i9")  
            .setRam("32GB")  
            .setStorage("2TB SSD")  
            .build();  
  
        c1.display();  
        c2.display();  
    }  
}
```

```
CPU: Intel i5  
RAM: 8GB  
Storage: 512GB SSD  
CPU: Intel i9  
RAM: 32GB  
Storage: 2TB SSD
```

Exercise 4: Implementing the Adapter Pattern

Scenario:

You are developing a payment processing system that needs to integrate with multiple third-party payment gateways with different interfaces. Use the Adapter Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **AdapterPatternExample**.
2. **Define Target Interface:**
 - Create an interface **PaymentProcessor** with methods like **processPayment()**.
3. **Implement Adaptee Classes:**
 - Create classes for different payment gateways with their own methods.
4. **Implement the Adapter Class:**
 - Create an adapter class for each payment gateway that implements **PaymentProcessor** and translates the calls to the gateway-specific methods.
5. **Test the Adapter Implementation:**
 - Create a test class to demonstrate the use of different payment gateways through the adapter.

<pre>// STEP 1: Project - AdapterPatternExample // STEP 2: Target Interface interface PaymentProcessor { void processPayment(double amount); } // STEP 3: Adaptee Class 1 class PayPalGateway { public void sendPayment(double amount) { System.out.println("PayPal Payment: " + amount); } } // STEP 3: Adaptee Class 2 class StripeGateway { public void makePayment(double amount) { System.out.println("Stripe Payment: " + amount); } } // STEP 4: Adapter for PayPal class PayPalAdapter implements PaymentProcessor { private PayPalGateway paypal; public PayPalAdapter(PayPalGateway paypal) { this.paypal = paypal; } public void processPayment(double amount) { paypal.sendPayment(amount); } } // STEP 4: Adapter for Stripe class StripeAdapter implements PaymentProcessor { private StripeGateway stripe; public StripeAdapter(StripeGateway stripe) { this.stripe = stripe; } public void processPayment(double amount) { stripe.makePayment(amount); } } // STEP 5: Test Adapter Pattern class Main { public static void main(String[] args) { PaymentProcessor p1 = new PayPalAdapter(new PayPalGateway()); PaymentProcessor p2 = new StripeAdapter(new StripeGateway()); p1.processPayment(1000); p2.processPayment(2000); } }</pre>	<pre>Output PayPal Payment: 1000.0 Stripe Payment: 2000.0 === Code Execution Successful</pre>
--	---

Exercise 5: Implementing the Decorator Pattern

Scenario:

You are developing a notification system where notifications can be sent via multiple channels (e.g., Email, SMS). Use the Decorator Pattern to add functionalities dynamically.

Steps:

Create a New Java Project:

Create a new Java project named **DecoratorPatternExample**.

Define Component Interface:

Create an interface **Notifier** with a method **send()**.

Implement Concrete Component:

Create a class **EmailNotifier** that implements **Notifier**.

Implement Decorator Classes:

Create abstract decorator class **NotifierDecorator** that implements **Notifier** and holds a reference to a **Notifier** object.

Create concrete decorator classes like **SMSNotifierDecorator**, **SlackNotifierDecorator** that extend **NotifierDecorator**.

Test the Decorator Implementation:

Create a test class to demonstrate sending notifications via multiple channels using decorators.

```

1 // STEP 1: Project - DecoratorPatternExample
2 // STEP 2: Component Interface
3 interface Notifier {
4     void send(String msg);
5 }
6 // STEP 3: Concrete Component
7 class EmailNotifier implements Notifier {
8     public void send(String msg) {
9         System.out.println("Email Notification: " + msg);
10    }
11 }
12 // STEP 4: Abstract Decorator
13 abstract class NotifierDecorator implements Notifier {
14     protected Notifier notifier;
15     public NotifierDecorator(Notifier notifier) {
16         this.notifier = notifier;
17     }
18 }
19 // STEP 4: SMS Decorator
20 class SMSNotifierDecorator extends NotifierDecorator {
21
22     public SMSNotifierDecorator(Notifier notifier) {
23         super(notifier);
24     }
25     public void send(String msg) {
26         notifier.send(msg);
27         System.out.println("SMS Notification: " + msg);
28     }
29 }

```

```

30 // STEP 4: Slack Decorator
31 class SlackNotifierDecorator extends NotifierDecorator {
32     public SlackNotifierDecorator(Notifier notifier) {
33         super(notifier);
34     }
35     public void send(String msg) {
36         notifier.send(msg);
37         System.out.println("Slack Notification: " + msg);
38     }
39 }
40
41 // STEP 5: Test Decorator Pattern
42 class Main {
43     public static void main(String[] args) {
44
45         Notifier notifier =
46             new SlackNotifierDecorator(
47                 new SMSNotifierDecorator(
48                     new EmailNotifier());
49
50         notifier.send("Payment Successful");
51     }
52 }
53

```

Output

```

Email Notification: Payment Successful
SMS Notification: Payment Successful
Slack Notification: Payment Successful

```

```

=== Code Execution Successful ===

```

Exercise 6: Implementing the Proxy Pattern

Scenario:

You are developing an image viewer application that loads images from a remote server. Use the Proxy Pattern to add lazy initialization and caching.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **ProxyPatternExample**.
2. **Define Subject Interface:**
 - Create an interface **Image** with a method **display()**.
3. **Implement Real Subject Class:**
 - Create a class **RealImage** that implements **Image** and loads an image from a remote server.
4. **Implement Proxy Class:**
 - Create a class **ProxyImage** that implements **Image** and holds a reference to **RealImage**.
 - Implement lazy initialization and caching in **ProxyImage**.
5. **Test the Proxy Implementation:**
 - Create a test class to demonstrate the use of **ProxyImage** to load and display images.


```

1 // STEP 1: Project - ProxyPatternExample
2 // STEP 2: Subject Interface
3 interface Image {
4     void display();
5 }
6 // STEP 3: Real Subject Class
7 class RealImage implements Image {
8     private String fileName;
9     public RealImage(String fileName) {
10         this.fileName = fileName;
11         loadFromServer();
12     }
13     private void loadFromServer() {
14         System.out.println("Loading " + fileName + " from remote server...");
15     }
16     public void display() {
17         System.out.println("Displaying " + fileName);
18     }
19 }
20 // STEP 4: Proxy Class
21 class ProxyImage implements Image {
22     private String fileName;
23     private RealImage realImage;
24
25     public ProxyImage(String fileName) {
26         this.fileName = fileName;
27     }
28     public void display() {
29         // Lazy Initialization
30         if (realImage == null) {
31             realImage = new RealImage(fileName);
32         }
33         // Cached Object Reuse
34         realImage.display();
35     }
36 }
37 // STEP 5: Test Proxy Pattern
38 class Main {
39     public static void main(String[] args) {
40         Image image = new ProxyImage("photo.jpg");
41         System.out.println("First Call:");
42         image.display();
43         System.out.println();
44         System.out.println("Second Call:");
45         image.display();
46     }
47 }

```

Output

```

First Call:
Loading photo.jpg from remote server...
Displaying photo.jpg

```

```

Second Call:
Displaying photo.jpg

```

```

=== Code Execution Successful ===

```

Exercise 7: Implementing the Observer Pattern

Scenario:

You are developing a stock market monitoring application where multiple clients need to be notified whenever stock prices change. Use the Observer Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **ObserverPatternExample**.
2. **Define Subject Interface:**
 - Create an interface **Stock** with methods to **register**, **deregister**, and **notify** observers.
3. **Implement Concrete Subject:**
 - Create a class **StockMarket** that implements **Stock** and maintains a list of observers.
4. **Define Observer Interface:**
 - Create an interface **Observer** with a method **update()**.
5. **Implement Concrete Observers:**
 - Create classes **MobileApp**, **WebApp** that implement **Observer**.
6. **Test the Observer Implementation:**
 - Create a test class to demonstrate the registration and notification of observers.

Main.java

```
1 // STEP 1: Project - ObserverPatternExample
2 import java.util.*;
3 // STEP 2: Subject Interface
4 interface Stock {
5     void registerObserver(Observer o);
6     void deregisterObserver(Observer o);
7     void notifyObservers();
8 }
9 // STEP 4: Observer Interface
10 interface Observer {
11     void update(double price);
12 }
13 // STEP 3: Concrete Subject
14 class StockMarket implements Stock {
15     private List<Observer> observers = new ArrayList<>();
16     private double stockPrice;
17     public void registerObserver(Observer o) {
18         observers.add(o);
19     }
20     public void deregisterObserver(Observer o) {
21         observers.remove(o);
22     }
23     public void notifyObservers() {
24         for (Observer o : observers) {
25             o.update(stockPrice);
26         }
27     }
28     public void setStockPrice(double stockPrice) {
29         this.stockPrice = stockPrice;
30         notifyObservers();
31     }
32 }
33 // STEP 5: Concrete Observer
34 class MobileApp implements Observer {
35     public void update(double price) {
36         System.out.println("Mobile App: Stock Price Updated to " + price);
37     }
38 }
39 // STEP 5: Concrete Observer
40 class WebApp implements Observer {
41     public void update(double price) {
42         System.out.println("Web App: Stock Price Updated to " + price);
43     }
44 }
45 // STEP 6: Test Observer Pattern
46 class Main {
47     public static void main(String[] args) {
48         StockMarket stockMarket = new StockMarket();
49         Observer mobile = new MobileApp();
50         Observer web = new WebApp();
51         stockMarket.registerObserver(mobile);
52         stockMarket.registerObserver(web);
53         stockMarket.setStockPrice(5000);
54     }
55 }
```

Output

```
▲ Mobile App: Stock Price Updated to 5000.0
  Web App: Stock Price Updated to 5000.0
```

```
=== Code Execution Successful ===
```

Exercise 8: Implementing the Strategy Pattern

Scenario:

You are developing a payment system where different payment methods (e.g., Credit Card, PayPal) can be selected at runtime. Use the Strategy Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **StrategyPatternExample**.
2. **Define Strategy Interface:**
 - Create an interface **PaymentStrategy** with a method **pay()**.
3. **Implement Concrete Strategies:**
 - Create classes **CreditCardPayment**, **PayPalPayment** that implement **PaymentStrategy**.
4. **Implement Context Class:**
 - Create a class **PaymentContext** that holds a reference to **PaymentStrategy** and a method to execute the strategy.
5. **Test the Strategy Implementation:**
 - Create a test class to demonstrate selecting and using different payment strategies.

Main.java

```
1 // STEP 1: Project - StrategyPatternExample
2 // STEP 2: Strategy Interface
3 interface PaymentStrategy {
4     void pay(double amount);
5 }
6 // STEP 3: Concrete Strategy
7 class CreditCardPayment implements PaymentStrategy {
8     public void pay(double amount) {
9         System.out.println("Paid " + amount + " using Credit Card");
10    }
11 }
12 // STEP 3: Concrete Strategy
13 class PayPalPayment implements PaymentStrategy {
14     public void pay(double amount) {
15         System.out.println("Paid " + amount + " using PayPal");
16    }
17 }
18 // STEP 4: Context Class
19 class PaymentContext {
20     private PaymentStrategy strategy;
21     public PaymentContext(PaymentStrategy strategy) {
22         this.strategy = strategy;
23     }
24     public void executePayment(double amount) {
25         strategy.pay(amount);
26     }
27 }
28 // STEP 5: Test Strategy Pattern
29 class Main {
30     public static void main(String[] args) {
31         PaymentContext payment1 =
32             new PaymentContext(new CreditCardPayment());
33         payment1.executePayment(1000);
34         PaymentContext payment2 =
35             new PaymentContext(new PayPalPayment());
36         payment2.executePayment(2000);
37     }
38 }
```

Output

```
Paid 1000.0 using Credit Card
Paid 2000.0 using PayPal
```

```
=== Code Execution Successful ===
```

Exercise 9: Implementing the Command Pattern

Scenario: You are developing a home automation system where commands can be issued to turn devices on or off. Use the Command Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **CommandPatternExample**.
2. **Define Command Interface:**
 - Create an interface **Command** with a method **execute()**.
3. **Implement Concrete Commands:**
 - Create classes **LightOnCommand**, **LightOffCommand** that implement **Command**.
4. **Implement Invoker Class:**
 - Create a class **RemoteControl** that holds a reference to a **Command** and a method to execute the command.
5. **Implement Receiver Class:**
 - Create a class **Light** with methods to turn on and off.
6. **Test the Command Implementation:**
 - Create a test class to demonstrate issuing commands using the **RemoteControl**.

Main.java

```
1 // STEP 1: Project - CommandPatternExample
2 // STEP 2: Command Interface
3 interface Command {
4     void execute();
5 }
6 // STEP 5: Receiver Class
7 class Light {
8     public void turnOn() {
9         System.out.println("Light is ON");
10    }
11    public void turnOff() {
12        System.out.println("Light is OFF");
13    }
14 }
15 // STEP 3: Concrete Command
16 class LightOnCommand implements Command {
17     private Light light;
18     public LightOnCommand(Light light) {
19         this.light = light;
20     }
21     public void execute() {
22         light.turnOn();
23     }
24 }
25 // STEP 3: Concrete Command
26 class LightOffCommand implements Command {
27     private Light light;
28     public LightOffCommand(Light light) {
29         this.light = light;
30     }
31     public void execute() {
32         light.turnOff();
33     }
34 }
35 // STEP 4: Invoker Class
36 class RemoteControl {
37     private Command command;
38     public RemoteControl(Command command) {
39         this.command = command;
40     }
41     public void pressButton() {
42         command.execute();
43     }
44 }
45 // STEP 6: Test Command Pattern
46 class Main {
47     public static void main(String[] args) {
48         Light light = new Light();
49         Command onCommand = new LightOnCommand(light);
50         Command offCommand = new LightOffCommand(light);
51         RemoteControl remote1 = new RemoteControl(onCommand);
52         RemoteControl remote2 = new RemoteControl(offCommand);
53         remote1.pressButton();
54         remote2.pressButton();
55     }
56 }
```

Output

Light is ON
Light is OFF

=== Code Execution Successful ===

Exercise 10: Implementing the MVC Pattern

Scenario:

You are developing a simple web application for managing student records using the MVC pattern.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **MVCPatternExample**.
2. **Define Model Class:**
 - Create a class **Student** with attributes like **name, id, and grade**.
3. **Define View Class:**
 - Create a class **StudentView** with a method **displayStudentDetails()**.
4. **Define Controller Class:**
 - Create a class **StudentController** that handles the communication between the model and the view.
5. **Test the MVC Implementation:**
 - Create a main class to demonstrate creating a **Student**, updating its details using **StudentController**, and displaying them using **StudentView**.


```

2 // STEP 2: Model Class
3 class Student {
4     private String name;
5     private int id;
6     private String grade;
7     public Student(String name, int id, String grade) {
8         this.name = name;
9         this.id = id;
10        this.grade = grade;
11    }
12    public String getName() {
13        return name;
14    }
15    public int getId() {
16        return id;
17    }
18    public String getGrade() {
19        return grade;
20    }
21    public void setName(String name) {
22        this.name = name;
23    }
24    public void setGrade(String grade) {
25        this.grade = grade;
26    }
27 }
28 // STEP 3: View Class
29 class StudentView {
30     public void displayStudentDetails(String name, int id, String grade) {
31         System.out.println("Student Details");
32         System.out.println("Name: " + name);
33         System.out.println("ID: " + id);
34         System.out.println("Grade: " + grade);
35     }
36 }
37 // STEP 4: Controller Class
38 class StudentController {
39     private Student model;
40     private StudentView view;
41     public StudentController(Student model, StudentView view) {
42         this.model = model;
43         this.view = view;
44     }
45     public void setStudentName(String name) {
46         model.setName(name);
47     }
48     public void setStudentGrade(String grade) {
49         model.setGrade(grade);
50     }
51     public void updateView() {
52         view.displayStudentDetails(
53             model.getName(),
54             model.getId(),
55             model.getGrade()
56         );
57     }
58 }

```

Output

```

Student Details
Name: Sionigdha
ID: 101
Grade: A

```

```

Student Details
Name: Snigdha
ID: 101
Grade: A+

```

=== Code Execution Successful ===

```

59 // STEP 5: Test MVC Pattern
60 class Main {
61     public static void main(String[] args) {
62         Student model =new Student("Sionigdha", 101, "A");
63         StudentView view =new StudentView();
64         StudentController controller =new StudentController(model, view);
65         controller.updateView();
66         controller.setStudentName("Snigdha");
67         controller.setStudentGrade("A+");
68         System.out.println();
69         controller.updateView();
70     }
71 }

```

Exercise 11: Implementing Dependency Injection

Scenario:

You are developing a customer management application where the service class depends on a repository class. Use Dependency Injection to manage these dependencies.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **DependencyInjectionExample**.
2. **Define Repository Interface:**
 - Create an interface **CustomerRepository** with methods like **findCustomerById()**.
3. **Implement Concrete Repository:**
 - Create a class **CustomerRepositoryImpl** that implements **CustomerRepository**.
4. **Define Service Class:**
 - Create a class **CustomerService** that depends on **CustomerRepository**.
5. **Implement Dependency Injection:**
 - Use constructor injection to inject **CustomerRepository** into **CustomerService**.
6. **Test the Dependency Injection Implementation:**
 - Create a main class to demonstrate creating a **CustomerService** with **CustomerRepositoryImpl** and using it to find a customer.

Main.java

```
1 // STEP 1: Project - DependencyInjectionExample
2 // STEP 2: Repository Interface
3 interface CustomerRepository {
4     String findCustomerById(int id);
5 }
6 // STEP 3: Concrete Repository
7 class CustomerRepositoryImpl implements CustomerRepository {
8     public String findCustomerById(int id) {
9         return "Customer ID: " + id + ", Name: Sionigdha";
10    }
11 }
12 // STEP 4: Service Class
13 class CustomerService {
14     private CustomerRepository repository;
15     // STEP 5: Constructor Injection
16     public CustomerService(CustomerRepository repository) {
17         this.repository = repository;
18     }
19     public void getCustomer(int id) {
20         System.out.println(repository.findCustomerById(id));
21     }
22 }
23 // STEP 6: Test Dependency Injection
24 class Main {
25     public static void main(String[] args) {
26         CustomerRepository repository = new CustomerRepositoryImpl();
27         CustomerService service = new CustomerService(repository);
28         service.getCustomer(101);
29     }
30 }
```

Output

Customer ID: 101, Name: Sionigdha

=== Code Execution Successful ===